Specification

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

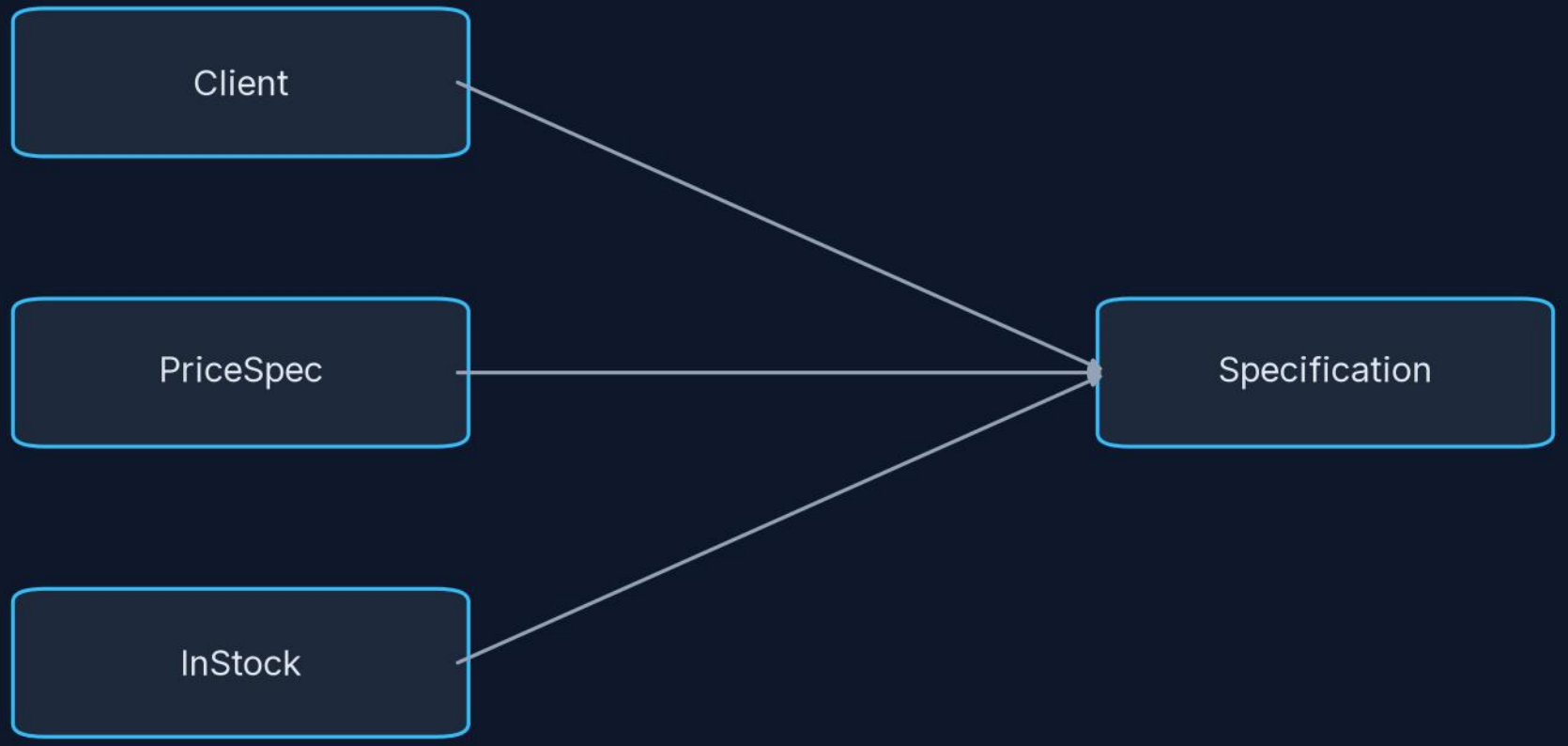
Today's Hook

- Today: Specification as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Specification structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Specification
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/specification/Specification.java
package com.buildwithwaqar.patterns.enterprise.specification;

import java.util.function.Predicate;

public interface Specification<T> extends Predicate<T> {
    default Specification<T> and(Specification<T> other) {
        return t -> this.test(t) && other.test(t);
    }

    default Specification<T> or(Specification<T> other) {
        return t -> this.test(t) || other.test(t);
    }

    default Specification<T> not() { return t -> !this.test(t); }
}

record Product(String name, int priceCents, boolean inStock) {}

final class PriceSpec implements Specification<Product> {
    private final int max;
    PriceSpec(int max) { this.max = max; }
    public boolean test(Product p) { return p.priceCents() <= max; }
}
```

Implementation (2)

```
final class InStockSpec implements Specification<Product> {
    public boolean test(Product p) { return p.inStock(); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.enterprise.specification;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Specification walkthrough ===");
    Specification<Product> rule = new PriceSpec(1000).and(new InStockSpec());
    for (Product p : new Product[]{new Product("mug", 500, true), new Product("mug", 500, false), new Product("tv", 50000, true)}) {
      System.out.println(p + " => " + rule.test(p));
    }
  }
}
```

Walkthrough Output

```
=== Specification walkthrough ===
Product[name=mug, priceCents=500, inStock=true] => true
Product[name=mug, priceCents=500, inStock=false] => false
Product[name=tv, priceCents=50000, inStock=true] => false
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Specification
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Specification → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.